Operating Systems: Hard Practice Questions

Midterm 1 · Chapters I–IV, Practice 1, labs · 40 questions · Answer key with explanations at the end
Hard on purpose: traces, calculations, code prediction and "why" questions. Questions tagged `beyond slides` go a little past the lecture material.

> **How to use:** try each question on paper, under exam conditions (about 90 minutes), before you look at the key. For the scenario questions, use the instructor's format: *process or thread → address space → mechanism → fix or trade-off*.

Part A — OS Concepts & System Programming (Ch. I)

1. A car's braking controller must respond within 2 ms, every time. A vendor proposes a general-purpose multi-user Linux with a *monolithic* kernel because "it's faster than a microkernel." Give two reasons the proposal is weak and say which OS type and kernel design you would choose.

2. Which of the following must involve a system call? (Select all that apply.)

A. `x = a * b + c` in C
B. `os.getpid()` in Python
C. Sorting a list already in memory
D. `open("log.txt","w")`
E. `platform.system()` the first time it runs
F. Calling a function defined in the same program

3. A buggy USB driver dereferences a NULL pointer. Describe what happens (a) on a monolithic kernel and (b) on a microkernel. Then explain why most desktop OSes still use monolithic or hybrid kernels anyway.

4. Your teammate says: "Python has `os` and `psutil`, so Python is a system programming language just like C." Write a 3-sentence rebuttal that uses at least **three** rows of the Python vs C table.

5. A system runs payroll overnight with no operator and also serves 200 terminal users during the day. Which **two** OS types from the slides does it combine? What conflict between them must the scheduler resolve?

Part B — Processes & Threads (Ch. II, Practice 1)

6. How many times is `hi` printed? Assume every `fork()` succeeds.

```
import os
for i in range(3):
    os.fork()
    print("hi")
```

7. How many processes exist in total, counting the original, after this C statement runs? Draw the process tree.

```
fork() && fork() || fork();
```

8. Predict the output of **both** programs and explain the difference in terms of address spaces.

```
# Program P                          # Program T
from multiprocessing import Process  from threading import Thread
counter = 0                          counter = 0
def work():                          def work():
    global counter                       global counter
    counter += 100                       counter += 100
p = Process(target=work)             t = Thread(target=work)
p.start(); p.join()                  t.start(); t.join()
print(counter)                       print(counter)
```

9. Two threads each run `counter += 1` 1,000,000 times on a shared `counter = 0`, with no lock. (a) What is the **maximum** final value? (b) What is the **minimum** possible final value? Give the interleaving that produces it. (c) Why does Python's GIL *not* prevent this?

10. `ps` shows 40 entries with state `Z` whose parent is PID 812, a long-running server. (a) What are they, and what resources do they still hold? (b) Why does `kill -9` on the zombies do nothing? (c) Give two fixes, one in the code and one from outside.

11. Process A is **Running**. Give the event that causes each transition, or write "impossible" and say why: (a) Running → Waiting (b) Waiting → Running (c) Running → Ready (d) Ready → Terminated (e) Waiting → Ready

12. A web server must handle 10,000 mostly idle keep-alive connections, plus some requests that run heavy image compression. Design it: where do you use processes, where threads, and why? Address memory use, crash isolation and CPU use on a multi-core machine.

13. With **many-to-one** user-level threads, one thread calls a blocking `read()` on a slow disk. What happens to the other 7 threads of that process, and why? Which thread model fixes this, and what does the fix cost?

14. Chrome runs each tab as a separate process. Give one advantage that would be **lost** if tabs were threads. Then give one cost Chrome pays for its design, and one way it reduces that cost.

15. Classify each workload as CPU-bound or I/O-bound and choose `multiprocessing` or `threading` in CPython:
(a) downloading 500 web pages (b) computing SHA-256 of 50 large files that are already in RAM (c) a GUI that polls `psutil.cpu_percent(interval=1)` (d) brute-forcing a 6-digit PIN against a local hash

16. In the week 3 dashboard, `psutil.cpu_percent(interval=1)` is called directly inside the Tkinter button callback, and the window freezes for 1 second on every refresh. Explain the freeze using the "frozen word processor" scenario, and give the fix.

Part C — Memory Management (Ch. III)

17. Fixed partitions of 100, 500, 200, 300 and 600 KB (in that order). Processes of 212, 417, 112 and 426 KB arrive in that order. Show where each process goes under **First-fit**, **Best-fit** and **Worst-fit** (treat a leftover piece as a new free hole). Which algorithm places all four processes?

18. Page size = 1 KB. Page table: page 0→5, 1→9, 2→2, 3→7, 4→3, 5→invalid. Translate each logical address to a physical address, or say what happens: (a) 2100 (b) 5000 (c) 4095 (d) 5200

19. A system has 32-bit logical addresses, 4 KB pages and 4-byte page-table entries.
(a) How many bits are offset, and how many are page number? (b) How many entries does a single-level page table have, and how big is it? (c) Why is that a problem with 100 processes, and what technique solves it? beyond slides
(d) Logical address `0x0003A7F2` is in a page that maps to frame `0x011`. What is the physical address?

20. Reference string: `7 0 1 2 0 3 0 4 2 3 0 3 2`, 3 frames that start empty. Count the page faults under **FIFO**, **LRU** and **Optimal**, and show the frame contents after each reference for LRU.

21. Reference string `1 2 3 4 1 2 5 1 2 3 4 5` with FIFO. Count the faults with 3 frames and with 4 frames. What is this phenomenon called? Why can LRU and Optimal never show it?

22. A buddy allocator manages 1024 KB. Requests arrive: A = 70 KB, B = 35 KB, C = 80 KB. (a) What block size does each get? (b) Total internal fragmentation? (c) List the free blocks after the three allocations. (d) A is freed: can it merge with anything? Why or why not?

23. For each technique, say which fragmentation it suffers from (internal, external or none), and explain why in one line: (a) fixed partitioning (b) dynamic partitioning (c) paging (d) segmentation (e) slab allocation

24. Memory access = 100 ns, TLB lookup = 10 ns, TLB hit ratio = 90%. Compute the effective access time. What hit ratio would you need to keep EAT ≤ 115 ns? beyond slides

25. A machine with 8 GB of RAM is running programs that need 14 GB in total. Disk activity is at 100%, CPU use is at 5%, and adding more programs makes it worse. Name the phenomenon, explain the feedback loop, and give two remedies.

26. In the paging lab, frames are filled in order and the page table is a Python dict. A student uses `page_table.pop(next(iter(page_table)))` to choose the victim. Which replacement policy is this, and why? What would you change to turn it into LRU?

27. A memory-leak detector samples `psutil.Process().memory_info().rss` every 5 s. RSS goes 120 → 180 → 150 → 210 → 190 → 260 MB. Is this a leak? Argue both ways, and say what a better detection rule would be.

Part D — File Systems & Directories (Ch. IV)

28. A 10-block file is stored with **contiguous**, **linked** and **indexed** allocation (assume the index block is already in memory). How many disk reads does it take to read block #7, the 8th block, with each method? How many to *append* a block at the end, if the FCB already holds the start and end pointers?

29. Block size = 4 KB and block pointers are 4 bytes. An inode has 12 direct pointers, 1 single-indirect pointer and 1 double-indirect pointer. What is the maximum file size? Show the three terms.

30. A 1 TB disk uses 4 KB blocks. How big is the free-space **bitmap**? Compare this with a free **linked list**: which one makes it easier to find 50 contiguous free blocks, and why?

31. In an **acyclic-graph** directory, `/a/report` and `/b/report` point to the same file. User A deletes `/a/report`. Describe two things that could go wrong and how reference counts and hard vs symbolic links handle each case.

32. Why do most file systems forbid hard links to **directories**? Explain in terms of the general-graph structure and what `du` or a recursive search would do.

33. Resolving `/home/zara/os/notes.md`: how many directory lookups are needed, starting from the root? If the current directory is `/home/zara`, how many are needed for `os/notes.md`? Why is this a reason for the current-working-directory concept?

34. A directory has 50,000 entries. Compare the cost of **lookup**, **create** (which must check for duplicates) and **ls in sorted order** with a linear list, a hash table and a B-tree. Which one would you choose for a mail server that stores one file per message?

35. Power fails while a file is being appended: the data block is written, but the inode and the free-space bitmap are not. (a) What inconsistent state results? (b) How does **metadata journaling** recover from it? (c) What can *still* be lost?

36. You have four 2 TB disks. For RAID 0, RAID 1 (two mirrored pairs) and RAID 5, give the usable capacity and how many disk failures each survives. Which one would you pick for a video-editing scratch disk, and which one for the university grades database?

37. `cat /proc/cpuinfo` shows a "file", but `du` reports 0 bytes for it. Explain this using the VFS layer and special file systems. Also explain why `tmpfs` data disappears on reboot.

38. Which file access method (sequential, direct or indexed) suits each case? (a) a tape backup restore, (b) a database fetching customer #48,213 by its fixed-size record number, (c) a dictionary looked up by word, (d) a streaming log reader.

Part E — Integration

39. Trace everything the OS does from the moment you type `python check-os.py` in Bash until the script's first `print` appears. Mention the shell, the `fork/exec` system calls, the process state changes, page faults during loading, and file-system path resolution.

40. *True or false? Justify each in one line.*
(a) Threads of the same process share their stacks.
(b) A larger page size reduces the page-table size but increases internal fragmentation.
(c) Virtual memory lets a process be larger than physical RAM.
(d) A directory structure and a file system are the same thing.
(e) An orphan process becomes a zombie forever.
(f) Contiguous allocation gives the best sequential *and* direct access, but suffers from external fragmentation.

Answer Key

Part A

1. A general-purpose multi-user OS optimizes for *throughput and fairness*, not guaranteed deadlines: its scheduling is not deterministic, so a 2 ms response is not guaranteed. A monolithic kernel also means one faulty driver can crash the whole controller. The slides say RTOSs "prioritize task scheduling" under strict timing constraints. **Choose:** a hard real-time embedded OS, ideally a microkernel such as QNX (or VxWorks), for fault isolation and predictable timing. "Faster on average" is not the same as "bounded worst case."

2. B, D, E. `getpid` asks the kernel for its process ID. `open` asks the file system. `platform.system()` calls `uname` or a Windows API underneath. A, C and F are pure user-mode computation.

3. (a) The driver runs in kernel space, so the fault is a kernel fault: a panic or blue screen, and the whole system goes down. (b) The driver is a user-space server process. Only that process crashes, and it can be restarted while the rest of the system keeps running. Desktops keep monolithic or hybrid kernels because microkernels pay for message passing (IPC and context switches) on every service request. Monolithic kernels are faster and have a mature driver ecosystem.

4. Example: Python is interpreted and slower, while C is compiled with direct hardware access, which kernels and real-time code need. C gives manual control of memory, which drivers require; Python hides it behind the interpreter. Python's `os`/`psutil` are wrappers around system calls that are implemented in C, so Python is great for *scripting and monitoring* (the slide's use case) but not for kernel or driver development.

5. Batch (jobs with no user interaction) and **multi-user** (time-sharing). The conflict: batch wants maximum throughput (long CPU bursts), while interactive users want short response times. The scheduler must give interactive jobs priority or short time slices, and run batch work in the background or off-hours.

Part B

6. 14. In iteration 0 there are 2 processes, which print 2. In iteration 1 there are 4, which print 4. In iteration 2 there are 8, which print 8. 2 + 4 + 8 = 14. At the end, 8 processes exist.

7. 5 processes. P0 forks C1. In P0, the first fork returned a value ≠ 0, so the `&&` evaluates the second fork and creates C2. P0 now has a nonzero result, so it skips `|| fork()`. C2 got 0, so it does `|| fork()` and creates C3. C1 got 0 from the first fork, so it skips the second and does `|| fork()`, creating C4. Tree: P0 → {C1 → C4, C2 → C3}. Total 1 + 4 = 5.

8. P prints **0**. T prints **100**. The child *process* gets its own copy of the address space (copy-on-write), so its change is invisible to the parent; sharing would need IPC (Queue, Value, Pipe). The *thread* shares the process's global data section, so the update is visible. This is Practice 1, Scenario 6.

9. (a) 2,000,000. (b) **2.** T1 loads 0 and is preempted. T2 runs 999,999 full iterations (counter = 999,999). T1 stores 1, wiping out T2's work. T2 loads 1 and is preempted. T1 runs its remaining 999,999 iterations. T2 stores 2. (c) += is several bytecodes (load, add, store). The GIL can switch threads *between* bytecodes, so the read-modify-write is not atomic. Fix: `threading.Lock`.

10. (a) Zombies: children that have exited but whose parent never called `wait()`. They hold no memory or CPU, only a process-table entry (PID and exit status). With enough of them, the system can run out of PIDs. (b) They are already dead, so there is nothing to kill; only the parent reaping them removes them. (c) In the code: call `wait()`/`waitpid()`, handle `SIGCHLD`, or `join()` in Python. From outside: kill or restart the parent (PID 812). Its zombies are then adopted by init/systemd, which reaps them.

11. (a) It requests I/O or waits for an event. (b) **Impossible:** it must go to Ready first and be picked by the scheduler. (c) Timer interrupt or preemption by a higher-priority process. (d) Normally impossible, because a process terminates while Running. The exception is being killed by its parent or the OS while waiting in Ready (an acceptable answer if justified). (e) The I/O or event it was waiting for completes.

12. Use threads (or an async event loop) for the 10,000 idle connections: they are cheap, share memory and suit I/O-bound work. Use a **pool of worker processes** (one per core) for image compression: it is CPU-bound, the GIL limits threads, and a crash in the codec kills only that worker. A process per connection would waste memory; threads alone would lack isolation and give no real parallelism in CPython. This is the trade-off in Practice 1, Scenario 4.

13. All 7 block. The kernel sees only one schedulable entity (the process), so a blocking system call blocks the whole process. The **one-to-one** (kernel-level threads) model fixes this. The cost is that every thread needs a kernel structure, and thread creation and switching need kernel involvement, so they are slower.

14. Lost: **isolation**. A crashing or malicious tab would bring down the whole browser, and threads share memory, so a compromised renderer could read another site's data. Cost: higher memory use, since each process has its own address space and copies of libraries. Mitigations: shared read-only code pages, a limit on renderer processes (sites can share a process), and discarding background tabs.

15. (a) I/O-bound, threading. (b) CPU-bound, multiprocessing. (c) I/O or sleep-bound, threading (and keep it off the GUI thread). (d) CPU-bound, multiprocessing.

16. `interval=1` blocks for 1 s. The callback runs on Tkinter's single main (UI) thread, so the event loop cannot redraw or handle clicks during that time. This is the same as the frozen word processor, whose UI thread was stuck on a long task. Fix: use `interval=None` (non-blocking) and schedule refreshes with `root.after()`, or do the sampling in a worker thread and pass the results back.

Part C

17. First-fit: 212 → 500 (hole 288), 417 → 600 (hole 183), 112 → 288 (hole 176), **426 waits**. **Best-fit:** 212 → 300, 417 → 500, 112 → 200, 426 → 600. All placed. **Worst-fit:** 212 → 600 (hole 388), 417 → 500, 112 → 388, **426 waits**. Only best-fit places all four in this case.

18. Physical address = frame × 1024 + offset. (a) 2100 = page 2, offset 52 → frame 2 → **2100** (this is a coincidence). (b) 5000 = page 4, offset 904 → frame 3 → **3976**. (c) 4095 = page 3, offset 1023 → frame 7 → **8191**. (d) 5200 = page 5, which is invalid → **trap**: a page fault if the page is on disk, or a segmentation fault if the address is illegal.

19. (a) 4 KB = 2^{12}, so 12 offset bits and 20 page-number bits. (b) 2^{20} = 1,048,576 entries × 4 B = **4 MB** per process. (c) 100 processes need 400 MB just for page tables, mostly for unused pages. Solutions: multi-level (hierarchical) page tables, inverted page tables or hashed page tables. (d) Page = 0x0003A, offset = 0x7F2 → frame 0x011 → physical **0x000117F2**.

20. FIFO = 10, **LRU = 9**, **Optimal = 7**. LRU trace (F = fault):

ref	7	0	1	2	0	3	0	4	2	3	0	3	2
frames	7	7 0	7 0 1	2 0 1	2 0 1	2 0 3	2 0 3	4 0 3	4 0 2	4 3 2	0 3 2	0 3 2	0 3 2
	F	F	F	F		F		F	F	F	F		

Optimal faults on: 7, 0, 1, 2 (evict 7), 3 (evict 1), 4 (evict 0), 0 (evict 4) = 7.

21. 3 frames: **9 faults**. 4 frames: **10 faults**. This is **Belady's anomaly**: more frames give more faults. LRU and Optimal are *stack algorithms*: the set of pages in memory with n frames is always a subset of the set with n+1 frames, so adding frames can never add faults. FIFO lacks this property.

22. (a) A gets 128 KB, B gets 64 KB, C gets 128 KB. (b) 58 + 29 + 48 = **135 KB**. (c) Split 1024 → 512 + 512, 512 → 256 + 256, 256 → 128(A) + 128, 128 → 64(B) + 64. C needs 128, so split the other 256 → 128(C) + 128. Free: **64, 128, 512**. (d) A's buddy is the 128 KB block that was split into B and the free 64. The buddy is not completely free, so A **cannot merge**. Merging needs the whole buddy to be free.

23. (a) Internal: the process is smaller than its fixed partition. (b) External: holes appear between processes. (c) Internal, only in the last page of each process; there is no external fragmentation because any frame fits. (d) External: segments vary in size. (e) Almost none: objects are exactly the slab's object size. There is a small amount of internal waste at the end of each slab.

24. EAT = 0.9(10 + 100) + 0.1(10 + 100 + 100) = 99 + 21 = **120 ns**. For ≤ 115: 110h + 210(1−h) ≤ 115 ⇒ 210 − 100h ≤ 115 ⇒ h ≥ **95%**.

25. Thrashing. The working sets do not fit, so processes page-fault constantly and wait on disk, and CPU use drops. An old-style scheduler sees a low-CPU system and admits more processes, which means fewer frames each and even more faults. Remedies: reduce the degree of multiprogramming (suspend or kill processes), add RAM, use working-set or page-fault-frequency control, or use local replacement.

26. FIFO. Python dicts keep insertion order, so the first key is the page that was loaded earliest. For LRU, move a page to the end on every *hit* as well (for example `OrderedDict.move_to_end(page)`) and evict the first item. The victim is then the least recently *used* page, not the oldest loaded one.

27. It could be a leak: the minimums climb (120 → 150 → 190) and so do the peaks. It might not be one: RSS rises and falls with caches, the garbage collector and allocator behaviour, and the series is short. A better rule: watch the *trend of the minimums* (or a moving average or regression slope) over many samples under a steady workload, and confirm with `tracemalloc`. Don't judge from one rise.

Part D

28. Reading block #7: **contiguous 1** (compute start + 7), **linked 8** (follow pointers through blocks 0–6, then read 7), **indexed 1** (the index is in memory). Appending: contiguous needs 1 write if the next block is free; otherwise the whole file must be relocated. Linked needs a write of the new block plus a rewrite of the old last block's pointer, about 2 I/Os, using the tail pointer. Indexed needs a write of the new block plus a write of the updated index block, about 2.

29. 4096 / 4 = 1024 pointers per block. Direct: 12 × 4 KB = 48 KB. Single indirect: 1024 × 4 KB = 4 MB. Double indirect: 1024^2 × 4 KB = 4 GB. Total ≈ **4 GB + 4 MB + 48 KB**.

30. 2^{40} / 2^{12} = 2^{28} blocks, so 2^{28} bits = **32 MB**. A bitmap makes it easy to find runs of contiguous free blocks: scan for 50 zero bits in a row, a word at a time. A linked list gives blocks in list order, which is not necessarily physical order, so finding contiguous space means walking the whole list. The list's advantage is that it costs no space when the disk is full.

31. (1) **Dangling pointer:** if the deletion freed the data, `/b/report` would point to freed (later reused) blocks. Hard links use a *reference count* in the inode, and the data is freed only when the count reaches 0, so `/b/report` keeps working. (2) With a **symbolic link** (a path string), deleting the target leaves a broken link. It fails safely ("No such file") rather than pointing to wrong data. A further subtlety: backups and traversals may visit a shared file twice.

32. Hard links to directories can create **cycles**, which turns the tree or DAG into a general graph. Recursive tools (`du`, `find`, backup) could loop forever or count data many times. Reference counts also fail: a cycle that is unreachable keeps a count above 0, so it needs garbage collection. Allowing only symbolic links (which traversal can skip) avoids this.

33. Absolute path: read `/`, find `home`, then `zara`, then `os`, then `notes.md`. That is **4 directory lookups** (/, home, zara, os), then the file's inode. Relative path from `/home/zara`: **2** (zara, os). The cwd saves repeated lookups from the root and gives shorter names.

34. Linear list: lookup O(n), create O(n) for the duplicate scan, sorted listing O(n log n). Hash table: lookup and create O(1) on average, but sorted listing O(n log n), plus collisions and the cost of resizing a fixed-size table. B-tree: lookup and create O(log n), sorted listing O(n) by an in-order walk, and it stays balanced as it grows. For a mail server, choose a hash table or a B-tree; a B-tree (as in ext4 HTree, NTFS, XFS) handles growth and sorted listing well. A linear list is too slow at 50,000 entries.

35. (a) The data block has been written, but no inode points to it and the bitmap still marks it free. The data is lost, and the block may be reused, which is harmless. In the reverse order (bitmap updated, inode not), a block is leaked. If the inode was updated first, it would point to garbage. (b) Before touching the real locations, the FS writes the metadata changes (inode and bitmap) to the journal and then a commit record. After a crash it *replays* committed transactions and discards uncommitted ones, so metadata is always consistent, with no full `fsck`. (c) In metadata-only mode, *file data* that was not yet flushed can be lost or stale. Ordered mode or full data journaling protects it, at a cost in performance.

36. RAID 0: 8 TB, survives **0** failures. RAID 1 (two mirrored pairs): 4 TB, survives 1 failure (up to 2 if they are in different pairs). RAID 5: **6 TB**, survives 1 failure (parity is spread across disks). Video scratch: RAID 0, for speed, since the data is re-creatable. Grades database: RAID 1 or RAID 5 (RAID 1 is often preferred for its write performance), *plus backups*. RAID is not a backup.

37. `/proc` is **procfs**, a special (pseudo) file system. The VFS gives it the same `open`/`read` interface as disk file systems, but its content is generated by the kernel at read time from its data structures. No disk blocks are used, so the size is reported as 0. `tmpfs` keeps its files in RAM (and swap), which is volatile memory, so they are gone at reboot.

38. (a) Sequential. (b) Direct (offset = n × record size). (c) Indexed (an index from key to location). (d) Sequential.

Part E

39. Bash reads the line. It resolves `python` through PATH, a directory search that is a series of path-resolution lookups in the FS. Bash calls **fork()**, and the child is a copy of the shell (New → Ready). The child calls **exec()**, which replaces its image with the Python interpreter: new code, data, heap and stack, with a page table but no pages loaded yet. The scheduler dispatches it (Ready → Running). Touching code and data causes **page faults** that load pages from disk (demand paging, Running → Waiting → Ready). Python `open()`s `check-os.py`, resolving its path through directory lookups to the inode and data blocks. It compiles and runs the script. `print` calls `write()` on stdout (fd 1, the terminal), which is a system call. Meanwhile Bash is blocked in `wait()`. When the script ends, the child exits and Bash reaps it, so no zombie is left.

40. (a) **False.** Each thread has its own stack and registers; they share code, data, heap and open files. (b) **True.** There are fewer pages to track, but more waste in the last page. (c) **True.** Only the working set needs to be in RAM; the rest is on disk. (d) **False.** The directory structure is the logical organization of names. The file system is the whole mechanism: allocation, free space, metadata and reliability. (e) **False.** init/systemd adopts it and reaps it when it exits. (f) **True.** Its fixed sizes and holes cause external fragmentation, and growing a file is hard.